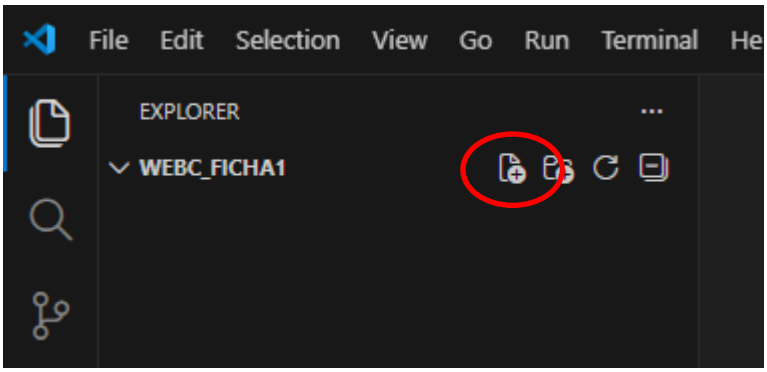
PRÁTICA LABORATORIAL 01

Objetivos:
- Criação e manutenção de um repositório Git;
- Ligação ao repositório online GitHub.

O git é um sistema de controlo e versionamento de projetos, permitindo a divisão de um projeto pelas pessoas que trabalham no mesmo e ainda controlar versões de desenvolvimento

PARTE I – CRIAÇÃO DO REPOSITÓRIO LOCAL

1. Crie uma pasta e atribua-lhe o nome "WEBC_Ficha1".
2. No Visual Studio Code abra o menu "File" e selecione "Open Folder". Escolha a pasta e clique no botão "Selecionar Pasta". Conseguirá perceber que é apresentada do lado esquerda uma zona com o nome da pasta.
3. Crie um ficheiro com o nome "README.md" dentro da pasta criada. Para isso deve clicar no botão "New File" junto ao nome da pasta, atribuir o nome ao ficheiro e carregar na tecla "Enter":



O ficheiro README.md é usado para apresentar informação sobre o repositório em que está inserido. Há normas específicas para formatação de um ficheiro deste tipo mas é algo que não vamos focar, ainda assim se quiser saber mais pode consultar o link: https://www.markdownguide.org/basic-syntax/

4. Acrescente o texto: "# Comandos de Git" ao ficheiro e grave.

Para aumentar a eficiência do trabalho pode selecionar a opção "Auto Save". No menu File selecionar a opção "Preferences" e depois "Settings" e na opções "Files: Auto Save" selecionar "onFocusChange", assim sempre que o foco alterar (mudança de ficheiro ou janela) o ficheiro é gravado.

5. Vamos agora criar o repositório. No menu "Terminal" selecione a opção "New Terminal" e verifique a abertura de uma janela na parte de baixo do editor.
6. Para verificar a correta instalação do git insira o comando "git --version" e verifique se é apresentada a informação sobre a versão instalada. Caso por alguma razão não seja apresentada a informação deve verificar se o git foi corretamente instalado na máquina de trabalho (é comum se instalou o git com o VS Code aberto ser necessário fechar o editor e voltar a abrir).
7. Para finalmente iniciar o repositório insira o comando "git init". Verifique se na mensagem de feedback há informação de criação do repositório. Repare que com a execução deste comando será apresentado ao lado do ficheiro "README.md" a letra "U", o que indica que o ficheiro não está a ser rastreado ("untracked").

Vamos neste ponto fazer uma configuração necessária na primeira utilização do git, que será a identificação do programador, assim no terminal execute os seguintes comandos trocando o nome e email do exemplo pelos seus dados.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git config --global user.name "Bruno"
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git config --global user.email "brunofrs7@gmail.com"
```

NOTA: este procedimento será necessário realizar apenas uma vez.

8. Crie agora um ficheiro com o nome “index.html” e insira como conteúdo “Ficheiro de HTML”.
9. Vamos agora verificar o estado do repositório. No terminal insira o comando “git status”. Verifique a mensagem:

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        index.html

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

É possível verificar que os dois ficheiros que criamos não estão a ser rastreados.

10. Adicione o ficheiro “index.html” ao controlo de versões utilizando o comando “git add index.html”. Execute novamente o “git status” e verifique que o ficheiro já está a ser rastreado.
11. Altere o conteúdo do ficheiro “README.md” acrescentando:

```
## Configuração de utilizador
- git config --global user.name "NOME"
- git config --global user.email "EMAIL"

## Comandos
- git init &#8594; inicia o repositório
- git status &#8594; verifica o estado do repositório
- git add NOMEFICHEIRO &#8594; adiciona um ficheiro específico ao repositório
```

Neste ponto o repositório local contém dois ficheiros, sendo que apenas o ficheiro HTML está a ser rastreado, assim o controlo de versões será realizado apenas nesse ficheiro.

PARTE II – CONTROLO DE VERSÕES NO REPOSITÓRIO

1. Vamos agora criar o primeiro ponto de controlo do nosso projeto, para isso vamos utilizar o comando "git commit". Sempre que o comando é executado é necessário adicionar uma mensagem que servirá para controlar o que foi feito desde a última versão. Assim execute o comando abaixo e verifique a mensagem de confirmação:

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git add index.html
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git commit -m "Adicionado ficheiro index.html"
[master (root-commit) 10b8ad6] Adicionado ficheiro index.html
 1 file changed, 1 insertion(+)
 create mode 100644 index.html
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

2. Executando o comando "git status" é possível verificar que o não há informação sobre o ficheiro "index.html" pois não foi alterado desde o último commit.
3. Altere o conteúdo do ficheiro "index.html" para "Ficheiro de HTML alterado".
4. Execute novamente o comando "git status" e verifique que é apresentada a informação de que o ficheiro "index.html"
5. Adicione o ficheiro "index.html" novamente para o controlo de versões e efetue novo commit com a mensagem "Ficheiro index.html alterado".

NOTA: pode utilizar o comando "git commit -am MENSAGEM" para adicionar os ficheiros alterados ao controlo de versões e commitar tudo no mesmo comando (evita a necessidade de fazer git add NOMEFICHEIRO).

6. Utilize o comando "git log" para verificar o histórico do repositório.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git log
commit e2c6e63666260e87b0ef72fdfba2032165e496c9 (HEAD -> master)
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:06:58 2026 +0000

    Ficheiro index.html alterado

commit 10b8ad67ef604e322ced339f704b0d1b150a7ba1
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:05:47 2026 +0000

    Adicionado ficheiro index.html
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

É possível verificar o código do commit (código a amarelo), autor, data e hora e mensagem informativa de cada commit, o que permite em cada momento verificar o que foi feito e por quem. De notar que neste caso como estamos a usar um repositório local só nosso o único autor apresentado será o utilizador atual, mas num repositório partilhado é essencial para perceber o trabalho desenvolvido por cada elemento.

Podemos ainda verificar, de forma mais simplificada, utilizando o comando git log –oneline

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git log --oneline
e2c6e63 (HEAD -> master) Ficheiro index.html alterado
10b8ad6 Adicionado ficheiro index.html
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

7. Para verificar o que foi atualizado entre versões é possível utilizar o comando “git show”. Copie o código do commit mais recente e execute o comando: “git show CODIGOCOMMIT”.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git show e2c6e63666260e87b0ef72fdfba2032165e496c9
commit e2c6e63666260e87b0ef72fdfba2032165e496c9 (HEAD -> master)
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:06:58 2026 +0000

    Ficheiro index.html alterado

diff --git a/index.html b/index.html
index ed66369..89601a7 100644
--- a/index.html
+++ b/index.html
@@ -1 +1 @@
-Ficheiro de HTML
\ No newline at end of file
+Ficheiro de HTML alterado
\ No newline at end of file
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

Como o git funciona com alteração de ficheiros por linha conseguimos verificar que no ficheiro index.html foi eliminada a linha que continha “Ficheiro de HTML” e adicionada a linha “Ficheiro de HTML alterado”. Efetivamente o que foi feito não foi a eliminação da linha e criação de uma nova, mas o funcionamento interno do git faz o controlo dessa forma.

NOTA: se executar apenas o comando “git show” são apresentadas as alterações do último commit.

8. Acrescente ao ficheiro “README.md” o seguinte conteúdo:

- git commit -m "MENSAGEM" → cria ponto no controlo de versões do repositório com os ficheiros adicionados - git commit -am "MENSAGEM" → cria um commit e adiciona os ficheiros modificados já adicionados a um commit anterior do repositório - git log → mostra as informações de todos os commit feitos - git show CODIGOCOMMIT→ apresenta as alterações efetuadas no commit do código fornecido - git show → apresenta as alterações efetuadas no último commit

PARTE III – FLUXOS DE TRABALHO COM BRANCH

Mais uma das funcionalidades muito interessantes e úteis é a possibilidade de criar fluxos de trabalho. Imaginando que no repositório está um projeto já terminado e em produção num cliente, mas que é necessário acrescentar uma nova funcionalidade. Caso estejamos a trabalhar no projeto e façamos uma alteração numa funcionalidade já existente pode acontecer de o programa deixar de funcionar, assim deve ser criado um fluxo de trabalho separado para não influenciar a versão atual do programa e quando tudo estiver implementado e testado passaremos para a versão em produção.

1. No terminal execute o comando “git branch novaFuncionalidade”
2. Execute agora o comando “git branch”. São apresentadas as duas linhas do tempo do repositório, neste caso a “master” que é a linha principal de qualquer repositório e a “novaFuncionalidade” que foi a que acabamos de acrescentar. Conseguimos ainda verificar que existe um * antes de “master” uma vez que é a branch que onde estamos.
3. Para mudar para a nova branch execute o comando “git checkout novaFuncionalidade” e verifique a mensagem de confirmação. Execute também um “git status” e verifique que somos informados de que estamos na branch “novaFuncionalidade”.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git branch novaFuncionalidade
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git branch
* master
  novaFuncionalidade
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git checkout novaFuncionalidade
Switched to branch 'novaFuncionalidade'
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git status
On branch novaFuncionalidade
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

4. Acrescente um novo ficheiro com o nome “ficheiro.txt” e conteúdo “Nova Funcionalidade do projeto”.
5. Adicione o ficheiro e realize o commit do mesmo com a mensagem “Nova Funcionalidade implementada”.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git status
On branch novaFuncionalidade
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        ficheiro.txt

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git add ficheiro.txt
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git commit -m "Nova funcionalidade implementada"
[novaFuncionalidade 6cd5b42] Nova funcionalidade implementada
 1 file changed, 1 insertion(+)
 create mode 100644 ficheiro.txt
```

6. Execute o comando “git log” e verifique que o último commit está relacionado com a branch “novaFuncionalidade” e que existe a branch “master” com o commit anterior.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git log
commit 6cd5b423bef8339476652411e4b4f382435775e4 (HEAD -> novaFuncionalidade)
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:12:35 2026 +0000

    Nova funcionalidade implementada

commit e2c6e63666260e87b0ef72fdfba2032165e496c9 (master)
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:06:58 2026 +0000

    Ficheiro index.html alterado

commit 10b8ad67ef604e322ced339f704b0d1b150a7ba1
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:05:47 2026 +0000

    Adicionado ficheiro index.html
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

7. Execute o comando “git checkout master” para voltar à branch principal e verifique o que acontece. A primeira alteração é que o ficheiro “ficheiro.txt” desapareceu, e realizando um “git log” não é possível verificar a existência do commit “Nova Funcionalidade implementada”. Isto deve-se ao facto de não estar presente nesta linha do tempo o que permite que não haja interferência de uma branch noutra.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git checkout master
Switched to branch 'master'
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git log
commit e2c6e63666260e87b0ef72fdfba2032165e496c9 (HEAD -> master)
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:06:58 2026 +0000

    Ficheiro index.html alterado

commit 10b8ad67ef604e322ced339f704b0d1b150a7ba1
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:05:47 2026 +0000

    Adicionado ficheiro index.html
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

NOTA: neste caso estamos a trabalhar com alteração de ficheiros diferentes, mas as alterações dentro de um ficheiro já existente também são consideradas.

8. Considerando que a nova funcionalidade está concluída e testado e pode ser passada para a branch “master” devemos fazer o “merge” das duas, aplicando o comando “git merge novaFuncionalidade”. É possível verificar que o “ficheiro.txt” é transportado para a branch “master” bem como qualquer outra alteração que pudesse ter sido feito na branch “novaFuncionalidade”.
9. Finalmente e como não vamos fazer mais alterações na branch “novaFuncionalidade” devemos alterar a mesma, assim é possível executar o comando “git branch -D novaFuncionalidade”. Execute o comando “git branch” para confirmar que só existe a branch “master”.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git merge novaFuncionalidade
Updating e2c6e63..6cd5b42
Fast-forward
 ficheiro.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 ficheiro.txt
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git log
commit 6cd5b423bef8339476652411e4b4f382435775e4 (HEAD -> master, novaFuncionalidade)
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:12:35 2026 +0000

    Nova funcionalidade implementada

commit e2c6e63666260e87b0ef72fdfba2032165e496c9
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:06:58 2026 +0000

    Ficheiro index.html alterado

commit 10b8ad67ef604e322ced339f704b0d1b150a7ba1
Author: Bruno <brunofrs7@gmail.com>
Date:   Mon Feb 16 12:05:47 2026 +0000

    Adicionado ficheiro index.html
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git branch -D novaFuncionalidade
Deleted branch novaFuncionalidade (was 6cd5b42).
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git branch
* master
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

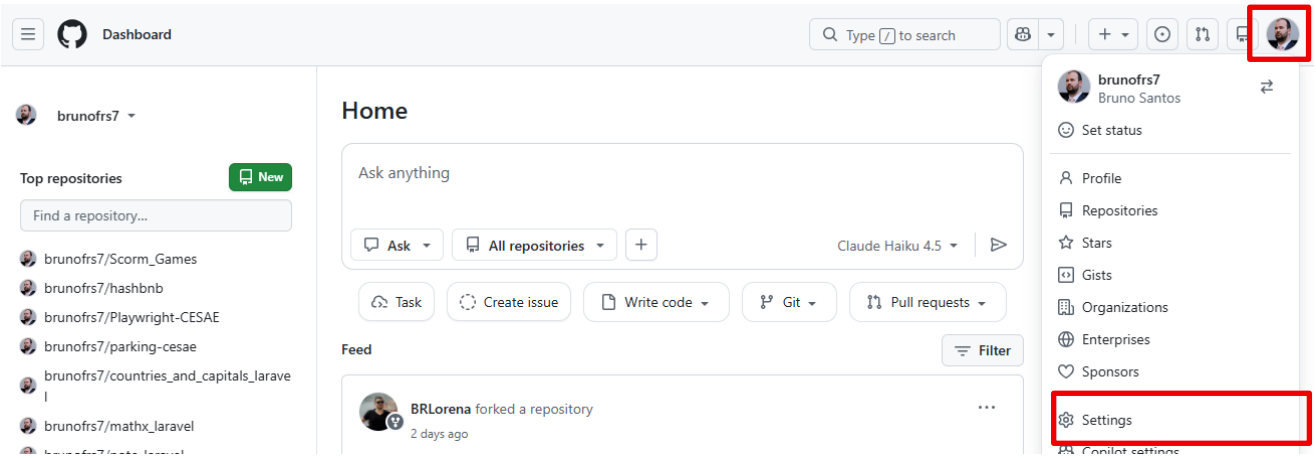
10. No ficheiro “README.md” acrescente o seguinte:

```
- git branch NOME &#8594; cria uma branch
- git branch &#8594; mostra as branch existentes
- git checkout NOME &#8594; troca para a branch NOME
- git merge NOME &#8594; estando numa branch acrescenta as funcionalidades da branch NOME à atual
- git branch -D NOME &#8594; elimina a branch NOME
```

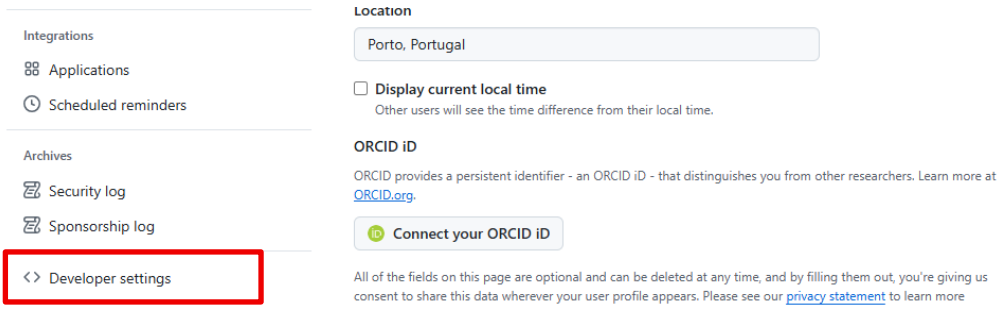
PARTE IV – UTILIZAÇÃO DO GITHUB

O GitHub é um dos repositórios online mais utilizados pela comunidade de desenvolvimento de software servindo não só para armazenamento e partilha de repositórios, mas também como rede social entre os desenvolvedores.

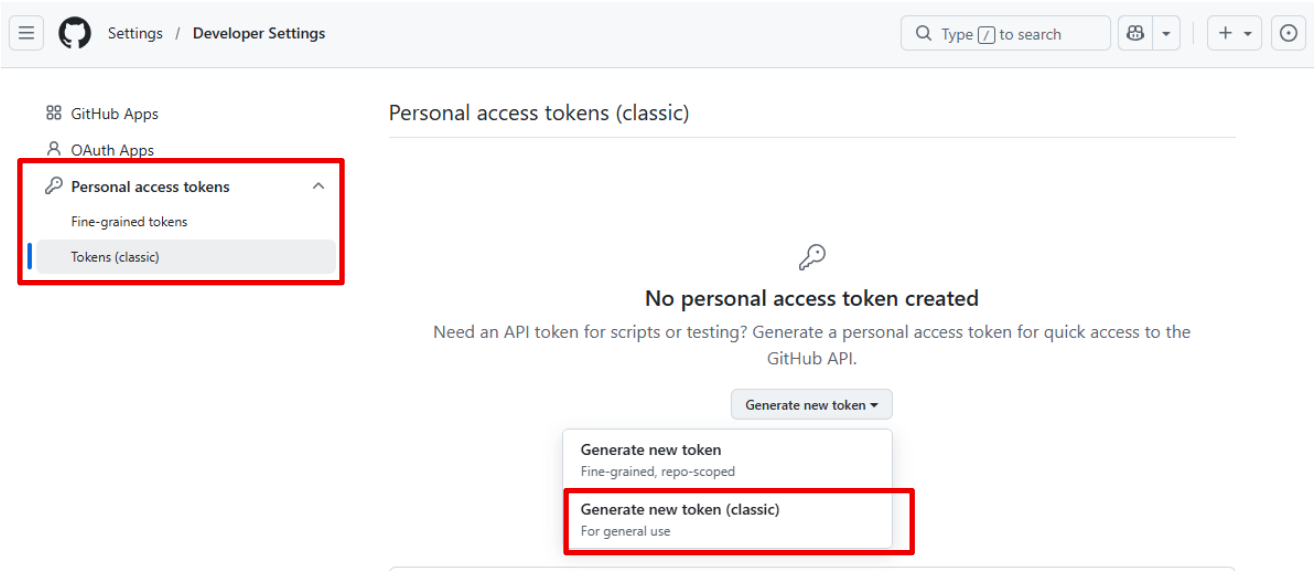
1. Registe-se no site https://github.com/
2. Após registo e autenticação será apresentada a sua área de trabalho. Antes de criar o repositório vamos gerar o token de acesso à conta, para isso vamos ao ícone de perfil no topo, a direita e selecionamos "Settings".



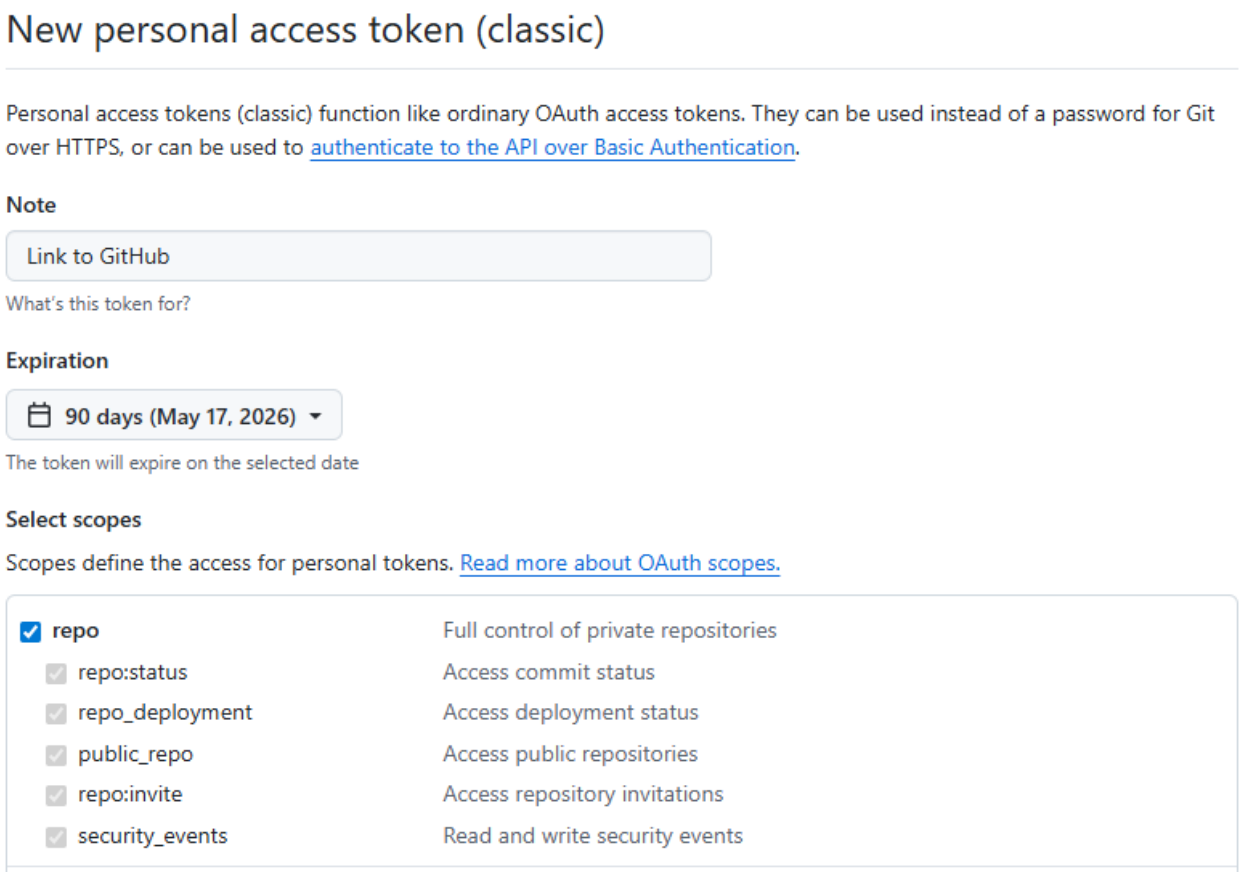
3. De seguida vamos abrir "Developer settings".



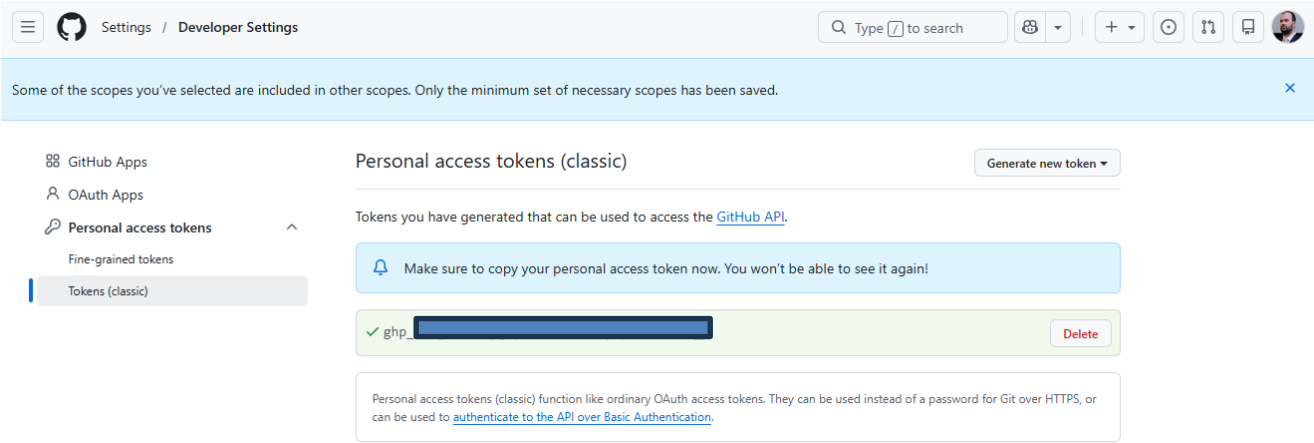
4. Em "Personal access tokens" a opção "Tokens (classic)" e de seguida "Generate new token (classic)"



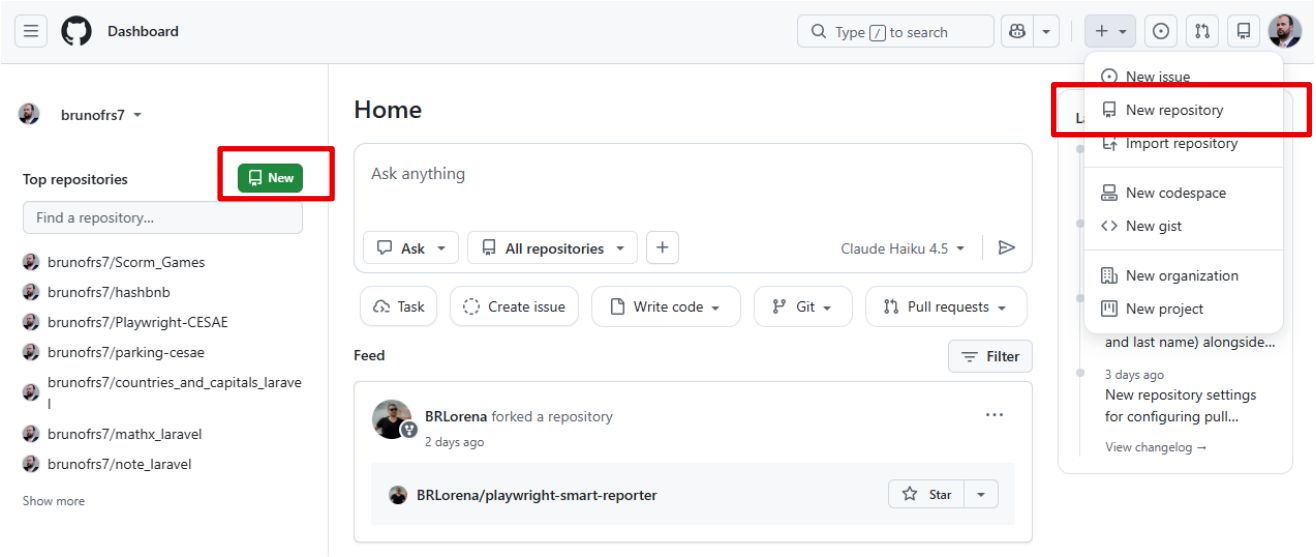
5. Damos um nome ao token, uma data de expiração e marcamos as opções de “repo”. Finalmente “Generate Token”.



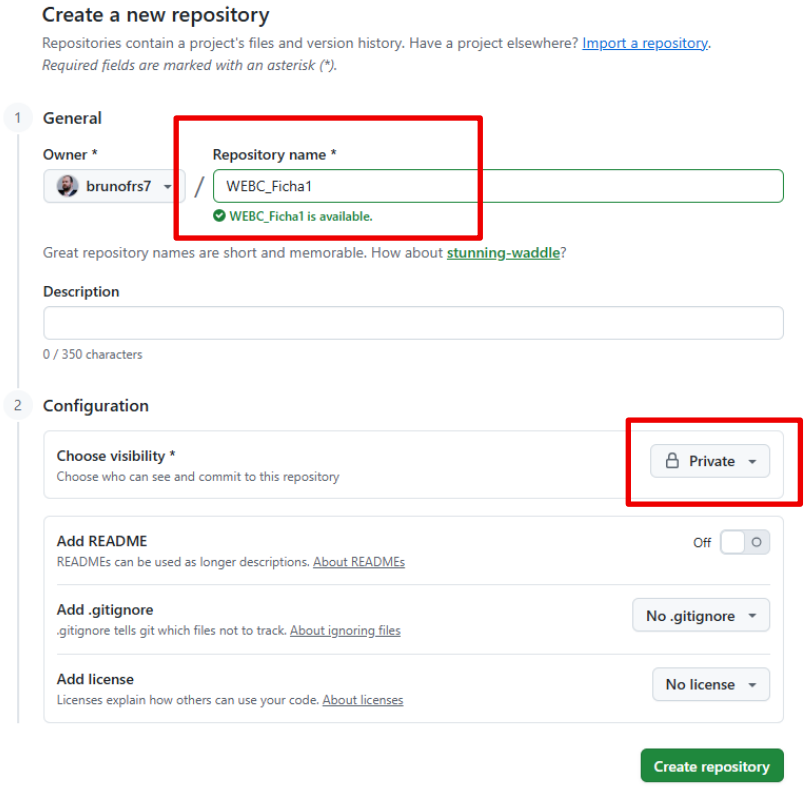
6. Gravamos o token apresentado antes de fechar a janela (o token não voltará a ser apresentado) e voltamos a página inicial.



7. Clique no botão verde à esquerda com a mensagem “New” ou no botão + em cima à direita e de seguida “New Repository” para criar um repositório online.

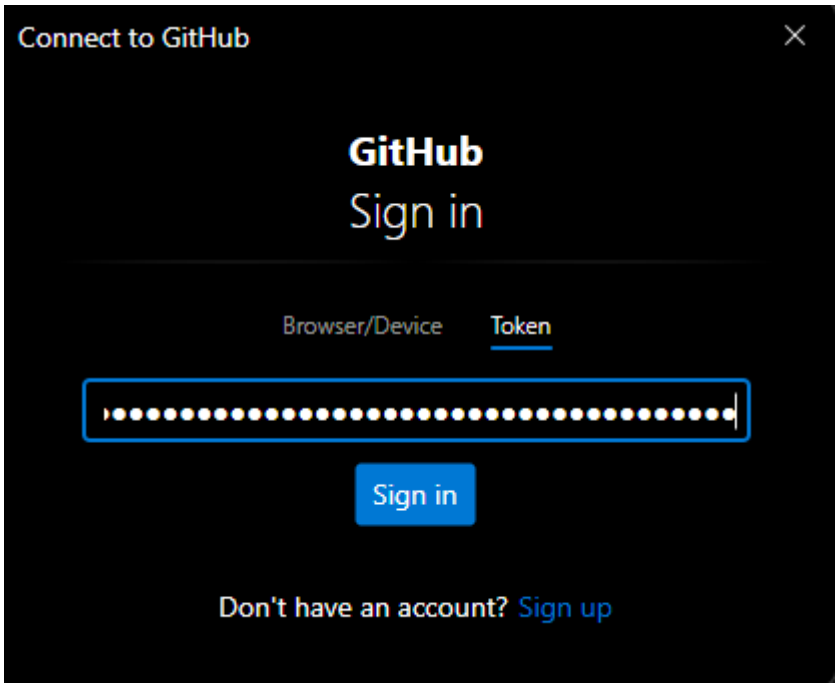


8. No campo “Repository name” coloque: “WEBC_Ficha1” e coloque-o como privado. Não marque a opção de criar um ficheiro README pois já temos esse ficheiro no nosso repositório privado e clique em “Create repository”.



9. Na nova janela são-nos apresentados dois blocos de código, sendo o primeiro “…or create a new repository on the command line” os comandos que devemos utilizar no terminal caso ainda não tenhamos um repositório local. No nosso caso vamos utilizar o bloco seguinte “…or push an existing repository from the command line”. Copie as três linhas abaixo e execute-as no terminal.

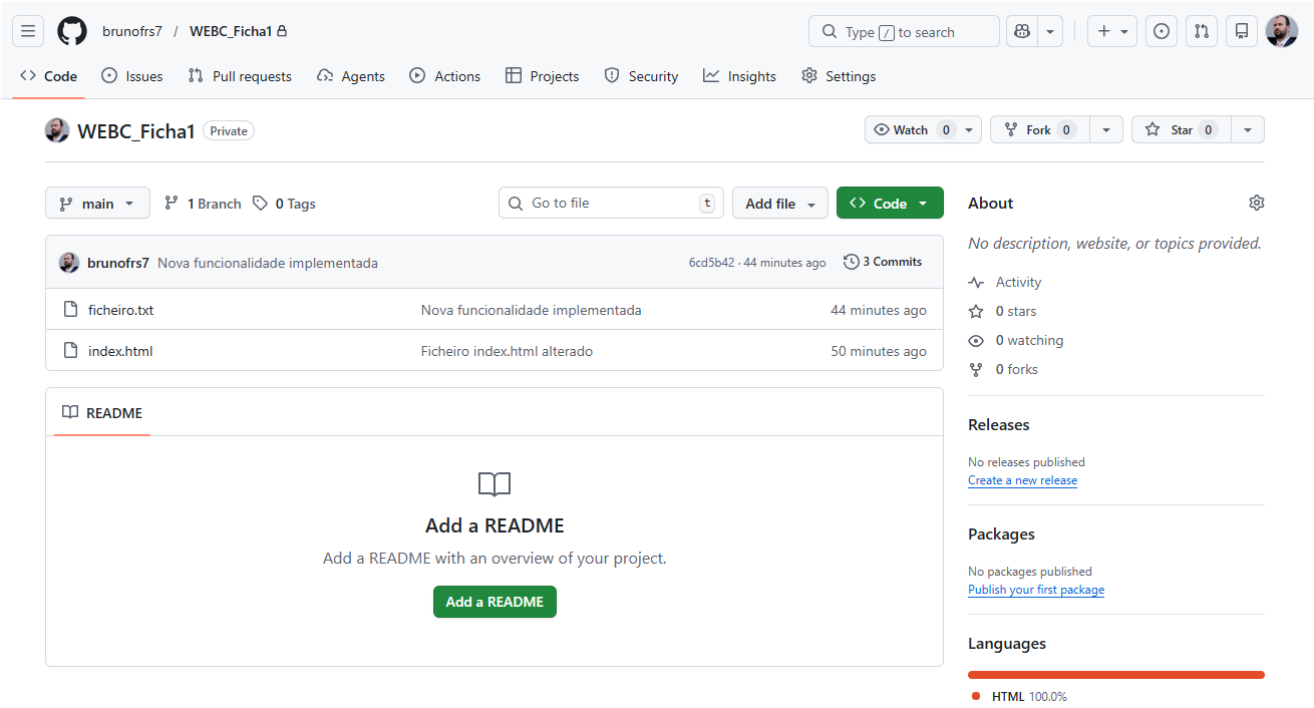
NOTA: deverá, na primeira execução, ser apresentada uma mensagem ou no terminal ou numa nova janela para o utilizador inserir as credenciais de acesso ao GitHub. A autenticação pode ser feita utilizando email/password, no entanto é mais fácil de gerir utilizando Token.




```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git remote add origin https://github.com/brunofrs7/WEBC_Ficha1.git
>> git branch -M main
>> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (9/9), 775 bytes | 775.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/brunofrs7/WEBC_Ficha1.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

O comando “git remote” vai permitir criar uma ligação entre o repositório atual local e o passado no link do repositório online.
O comando “git push” permite enviar o repositório atual local para o GitHub. De notar que na primeira execução é necessário especificar a branch master para que a mesma seja criada no repositório online. Posteriormente bastará utilizar o comando “git push” para atualizar o projeto no GitHub.
NOTA: caso de cada vez que executa um “git push” seja obrigado a inserir as suas credenciais do GitHub pode utilizar o comando “git config credential.helper store” para armazenar os dados de acesso na máquina em que está a trabalhar.

10. Neste ponto temos o repositório local ligado ao repositório online. No browser clicando na aba “Code” é possível verificar que já estão online os ficheiros do último commit.



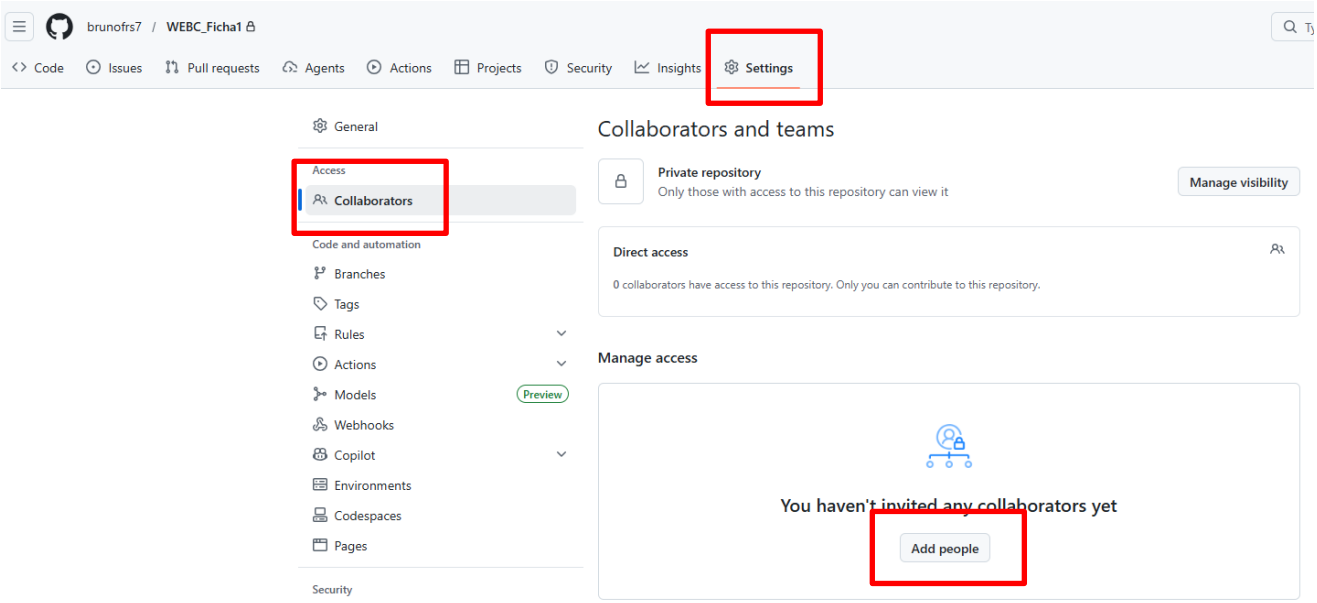
11. No ficheiro “README.md” acrescente o seguinte:

- git remote add origin LINK → liga o repositório atual local a um repositório online - git push -u origin master → envia a branch master para o repositório online (necessário apenas no primeiro envio) - git push → enviar o repositório atual local para o repositório online

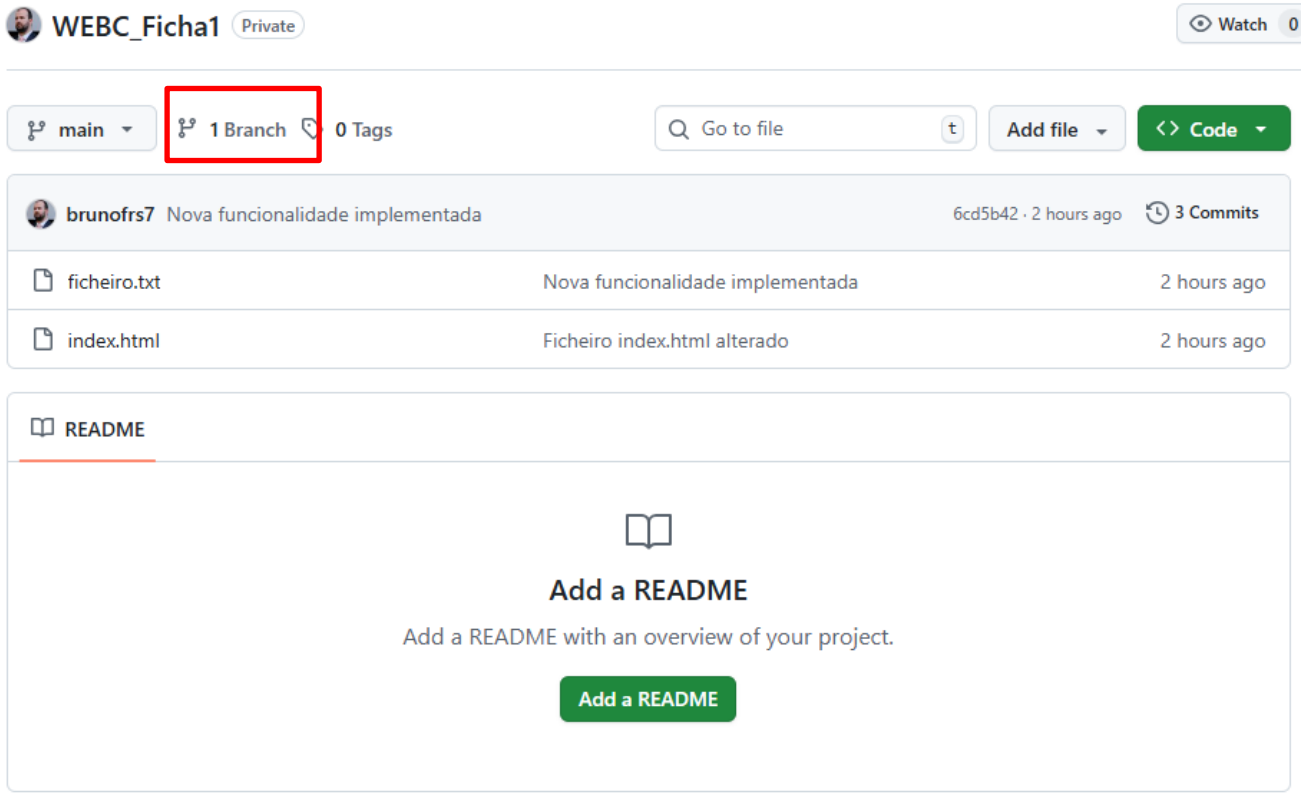
PARTE V – TRABALHO COLABORATIVO

O trabalho colaborativo entre membros da equipa é um fator essencial no que respeita à utilização do Git e GitHub. Vamos considerar um projeto a ser partilhado entre vários colaboradores, onde cada colaborador terá a sua branch e responsabilidade de implementar uma parte do projeto.

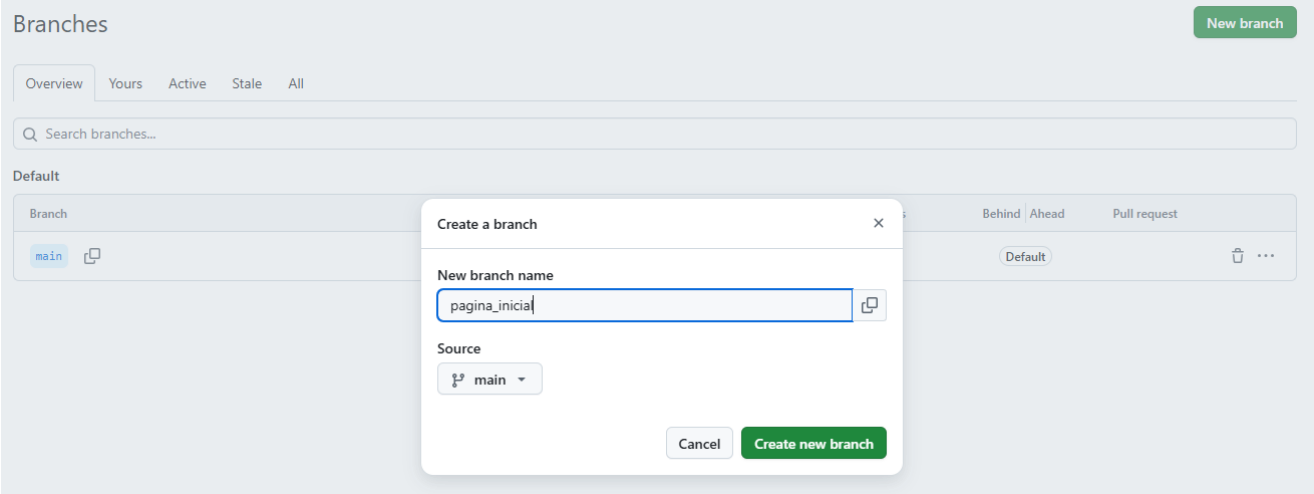
1. Dentro do repositório vamos a aba "Settings" e em "Collaborators" selecionar "Add people" e escolher as pessoas que vão trabalhar neste repositório.



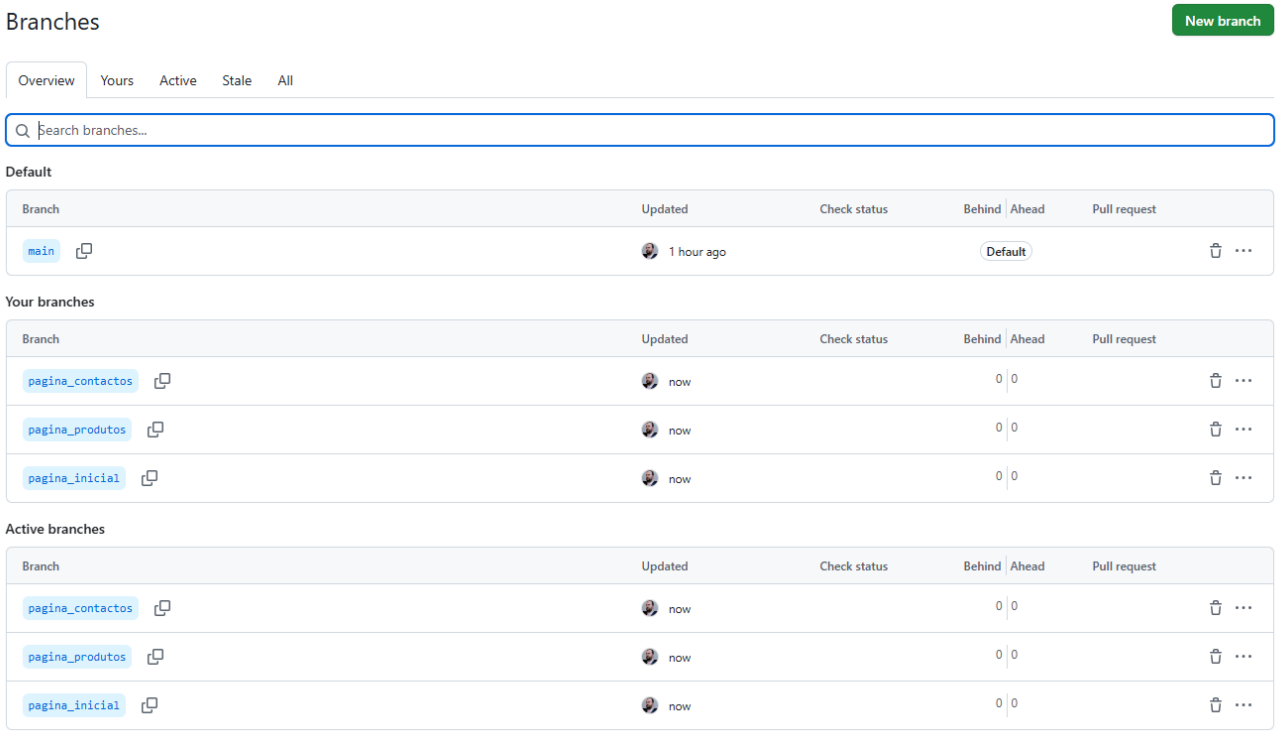
2. De seguida, vamos criar as várias branch para os colaboradores, este processo pode ser feito no GitHub ou no Git, sendo necessário sincronizar os dois repositórios (remoto e local). Neste caso vamos criar no GitHub, selecionando "Branch".



3. Clicar em “New branch” e inserir o nome da mesma, com “Source” na “main”. Uma vez que a “main” será a branch principal, na qual ficará guardada sempre a versão mais atualizada e completa, será sempre a partir desta que criaremos branches.



4. Repetimos o processo para mais 2 branches.



5. Com o comando git branch conseguimos verificar que localmente só temos a branch main. Para carregar as branches para o repositório local vamos usar o comando git fetch.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git branch
* main
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git fetch
From https://github.com/brunofrs7/WEBC_Ficha1
 * [new branch]      pagina_contactos -> origin/pagina_contactos
 * [new branch]      pagina_inicial   -> origin/pagina_inicial
 * [new branch]      pagina_produtos  -> origin/pagina_produtos
```

Notar que ao fazer o fetch o que acontece é recebermos a informação de que as branch existem remotamente, mas apenas se mudarmos para alguma delas irá passar a estar disponível localmente.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git branch
* main
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git checkout pagina_contactos
branch 'pagina_contactos' set up to track 'origin/pagina_contactos'.
Switched to a new branch 'pagina_contactos'
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git branch
  main
* pagina_contactos
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git branch
* main
  pagina_contactos
```

Se quisermos ver todas as branch usamos o comando git branch -a

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git branch -a
* main
  pagina_contactos
  remotes/origin/main
  remotes/origin/pagina_contactos
  remotes/origin/pagina_inicial
  remotes/origin/pagina_produtos
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

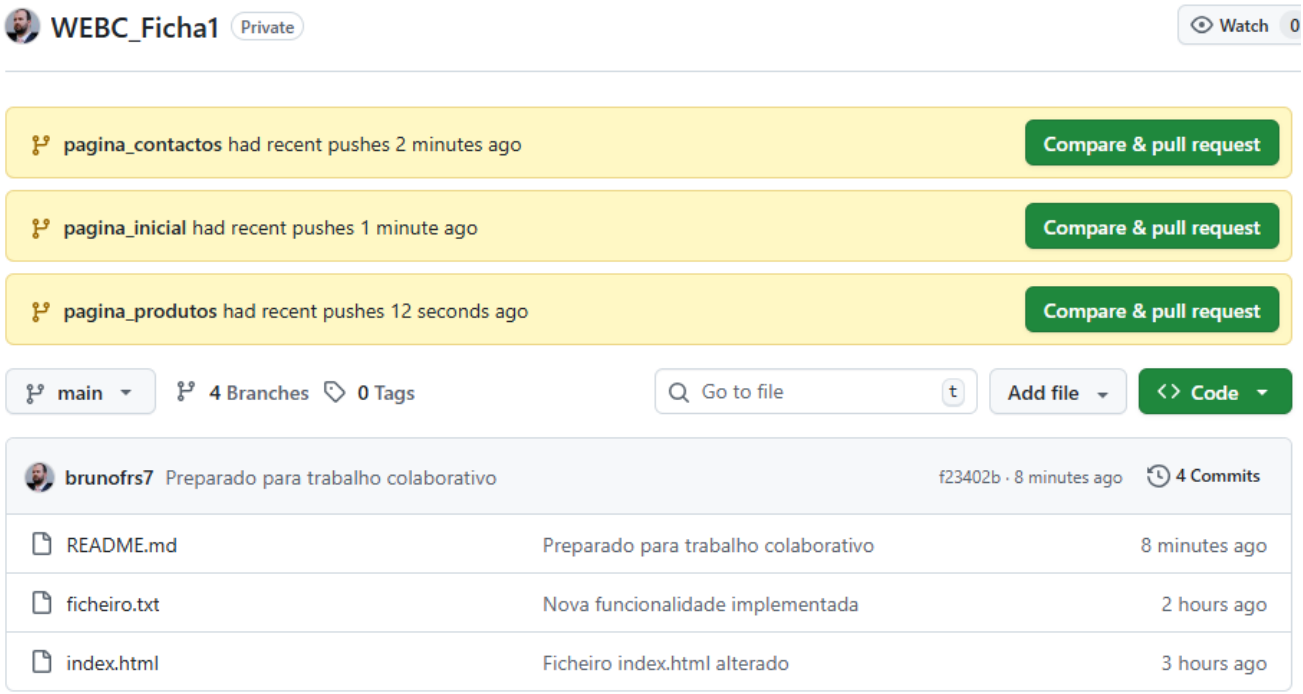
6. Na branch main vamos adicionar todos os ficheiros (README.md ainda não estava a ver rastreado) executar um commit e enviar as alterações para o GitHub com git push.

```
WEBC_FICHA1
  ficheiro.txt
  index.html
  README.md

PS C:\Users\bruno\Desktop\WEBC_Ficha1> git add .
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git commit -m "Preparado para trabalho colaborativo"
[main f23402b] Preparado para trabalho colaborativo
 1 file changed, 23 insertions(+)
 create mode 100644 README.md
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 902 bytes | 902.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/brunofrs7/WEBC_Ficha1.git
   6cd5b42..f23402b  main -> main
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

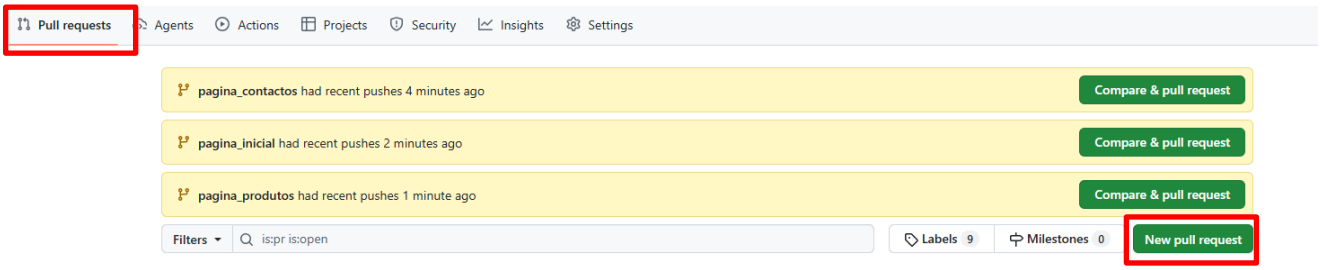
7. Trocamos para cada uma das branch, adicionamos o novo ficheiro representativo da branch (pagina_contactos → contactos.html; pagina_inicial → inicio.html; pagina_produtos →produtos.html), realizamos o commit e enviamos para o GitHub com push.

```
EXPLORER
WEBC_FICHA1
  contactos.html
  ficheiro.txt
  index.html

contactos.html
1   Página de contactos

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   powershell

PS C:\Users\bruno\Desktop\WEBC_Ficha1> git checkout pagina_contactos
Switched to branch 'pagina_contactos'
Your branch is up to date with 'origin/pagina_contactos'.
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git add .
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git commit -m "Página de contactos adicionada"
[pagina_contactos f90d553] Página de contactos adicionada
 1 file changed, 1 insertion(+)
 create mode 100644 contactos.html
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 350 bytes | 350.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/brunofrs7/WEBC_Ficha1.git
   6cd5b42..f90d553  pagina_contactos -> pagina_contactos
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

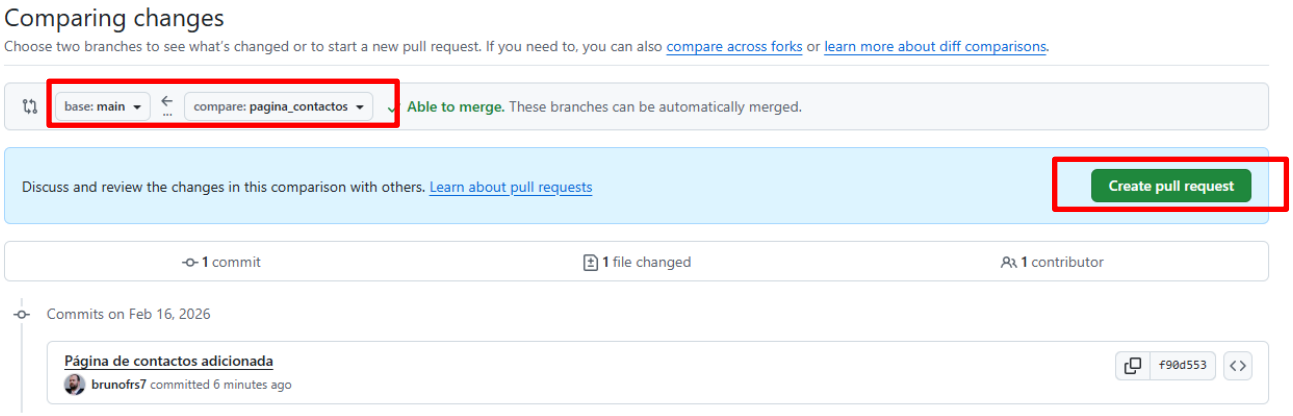
8. Após este processo, abrindo o GitHub (como um dos colaboradores) conseguimos perceber que temos informação de novos pushes dentro das branch, os quais devem gerar pull request.



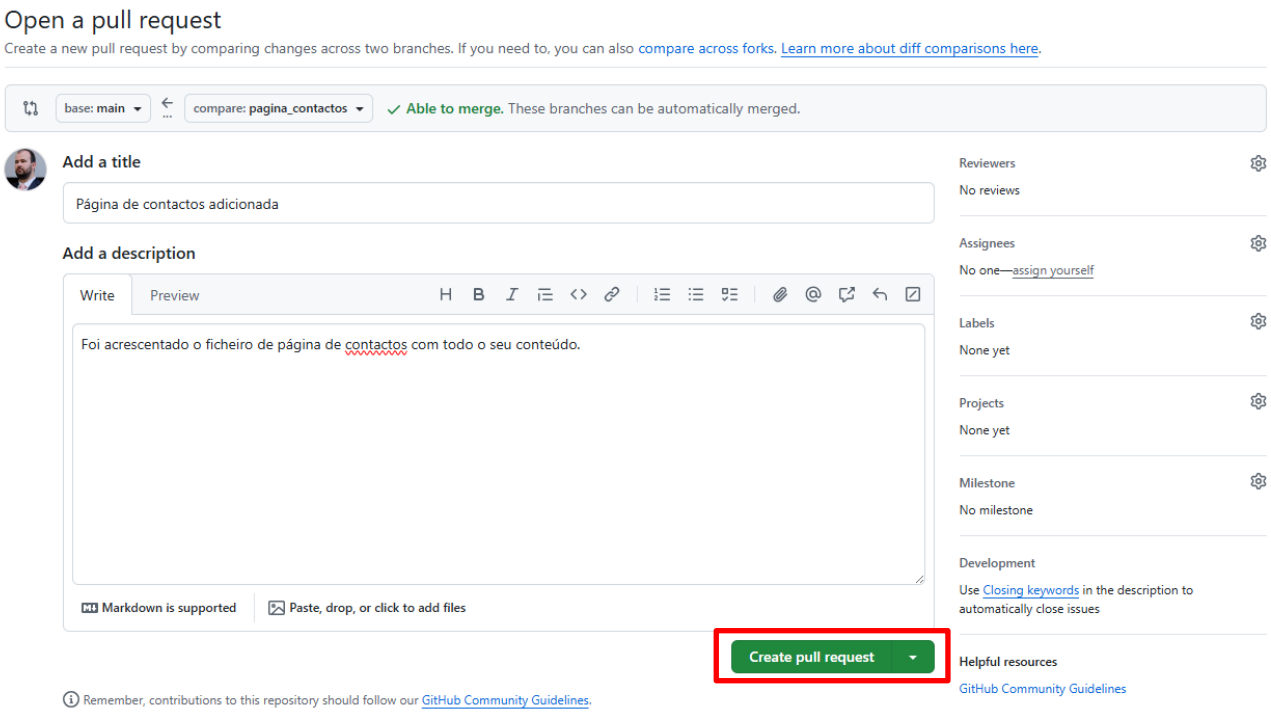
9. Clicando na aba de “Pull Request” vamos criar um “New pull request”.



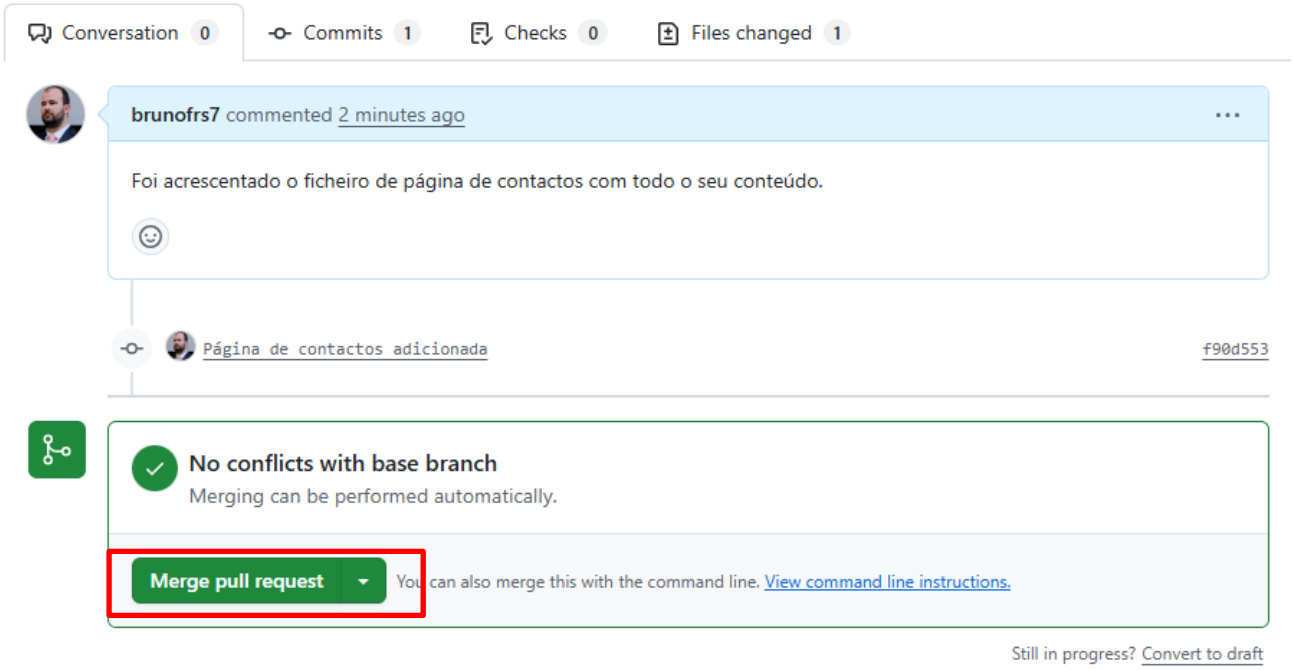
10. Selecionamos que queremos juntar a informação de uma das branch página para a branch main e "Create pull request".



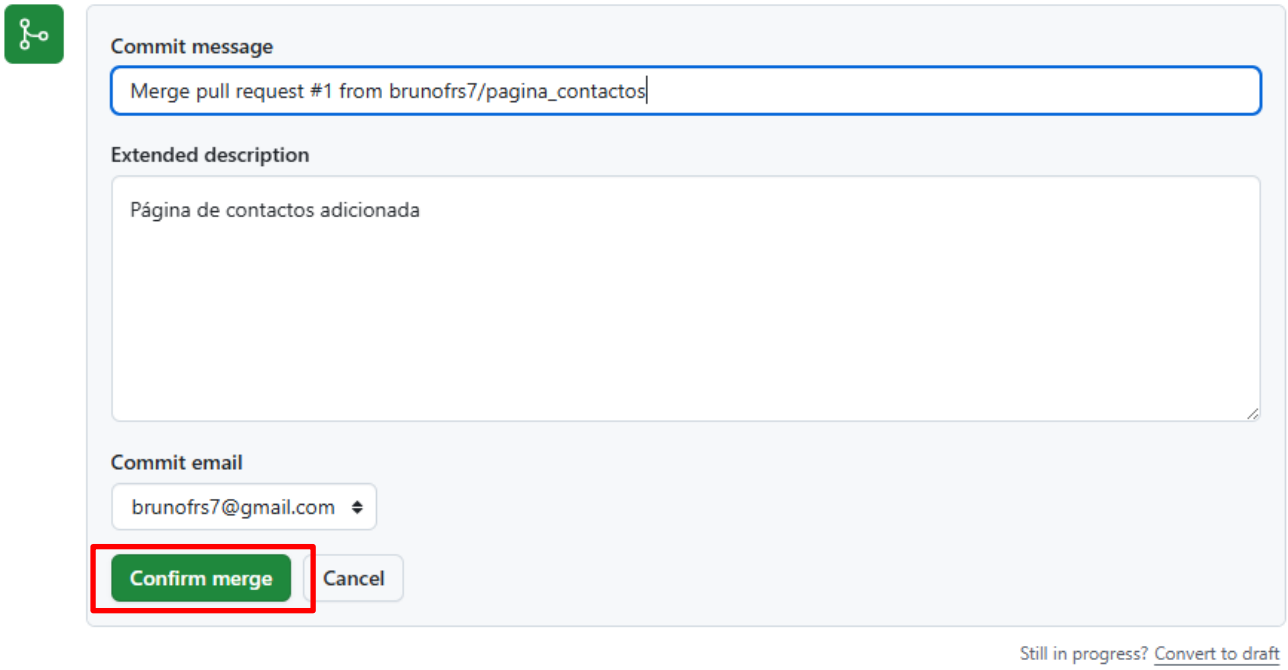
11. Aqui deve ser apresentada uma descrição do que foi feito de forma a ser analisada e validada pelo administrador do repositório. Finalmente "Create pull request".



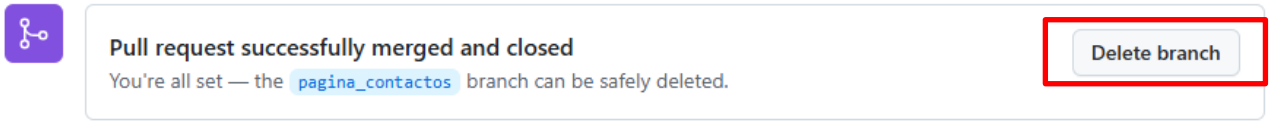
12. Como administrador do repositório devo abrir o mesmo verificar o pull request que foi criado e validar se o mesmo é aceite ou não. Posso colocar comentários durante o processo e/ou após a decisão de aceitação ou recusa. Se tudo estiver de acordo selecionamos "Merge pull request", caso contrário podemos rejeitar em definitivo em "Close pull request". Neste caso vamos aceitar.



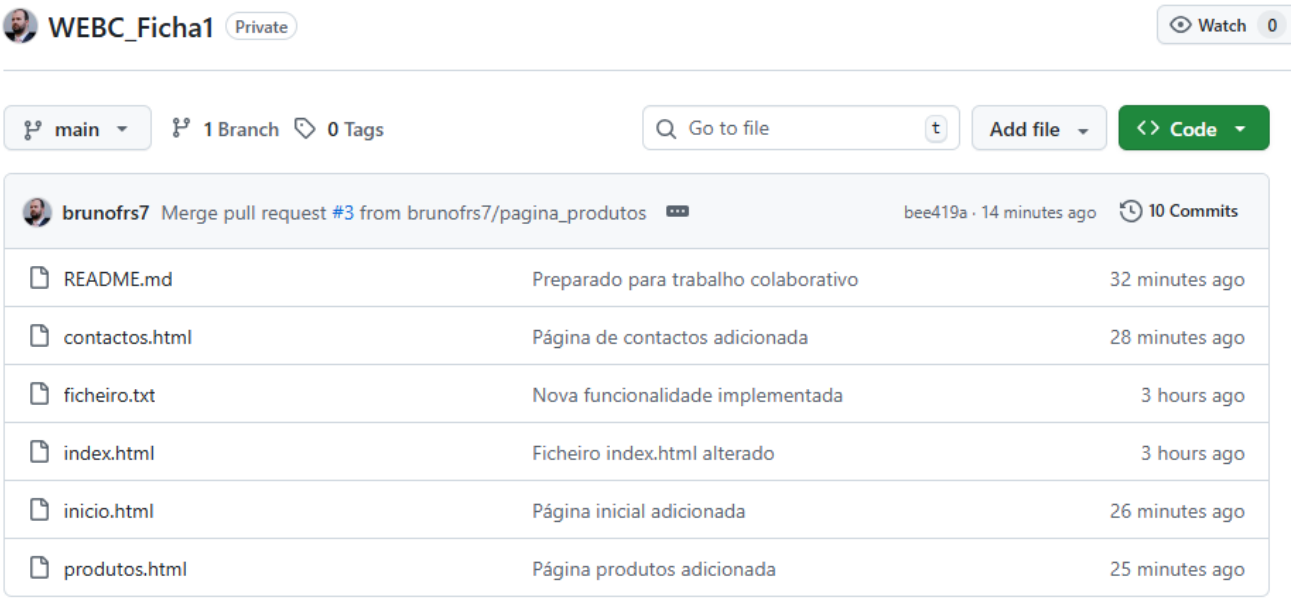
13. Confirmar merge.



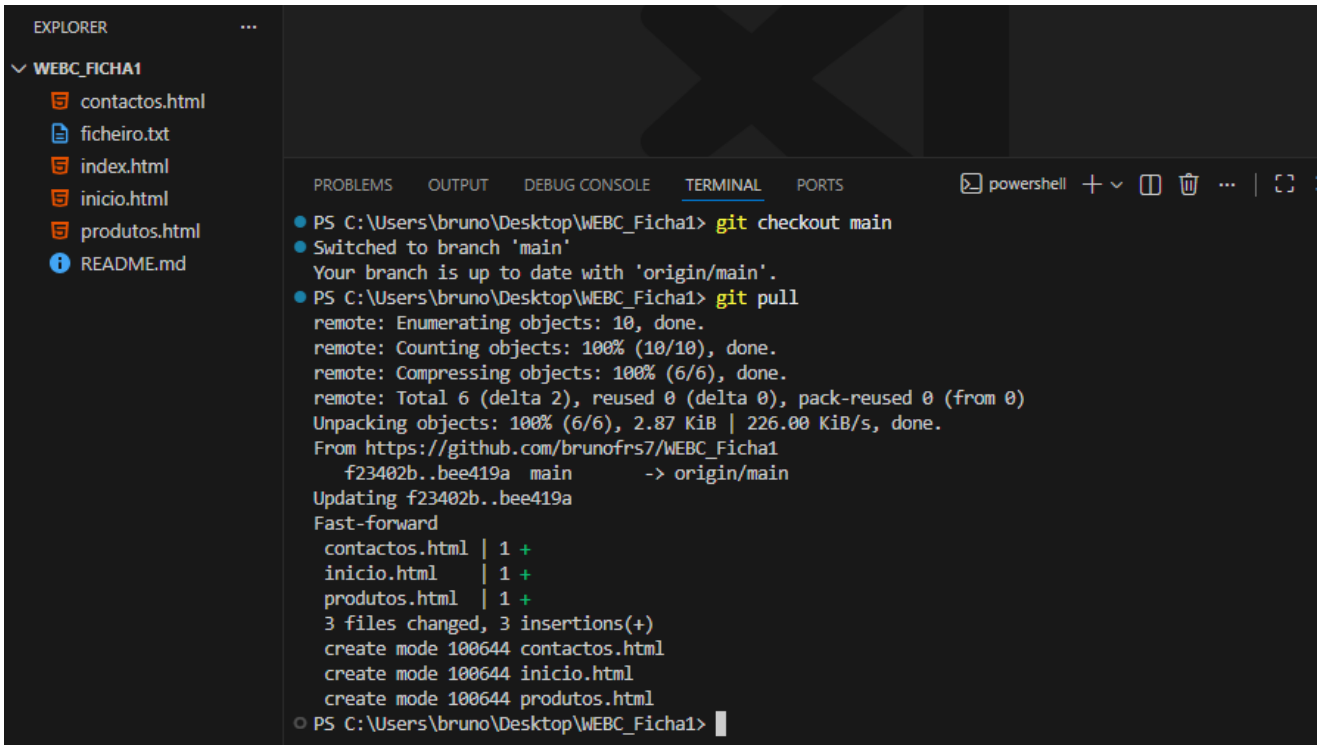
14. Finalmente podemos eliminar a branch caso este já não faça sentido existir.



15. Fazendo o mesmo processo com todas as branch temos o repositório completo.



16. Localmente voltamos a branch main e fazemos git pull para atualizar o repositório local com as atualizações remotas.



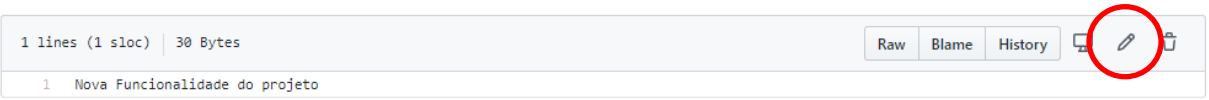
17. No ficheiro “README.md” acrescente o seguinte:

- git fetch → carrega as branch remotas para o repositório local - git branch -a → mostra todas as branch (locais e remotas) - git push → enviar o repositório atual local para o repositório online - git pull → descarregar alterações que existam no repositório online

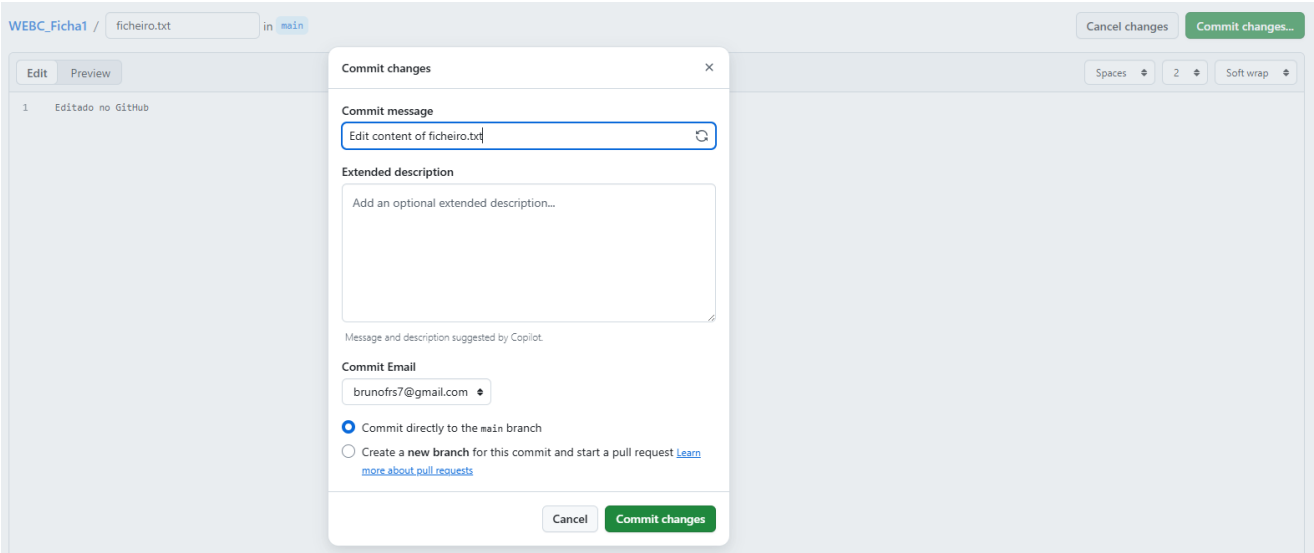
PARTE VI – RESOLUÇÃO DE CONFLITOS

Se o repositório em utilização for partilhado por vários contribuidores ou a mesma pessoa utilize duas máquinas diferentes para realizar operações, é possível que em algum momento haja alterações conflituosas. Por exemplo, o utilizador A eliminou o "ficheiro.txt" e o utilizador B alterou o conteúdo do "ficheiro.txt". Neste caso tem de haver um ponto de entendimento entre ambas as partes.

1. Clique no "ficheiro.txt" e verifique o seu conteúdo. Clique agora o símbolo de edição à direita e edite o ficheiro para "Editado no GitHub".



2. Após edição clique no botão verde "Commit changes".

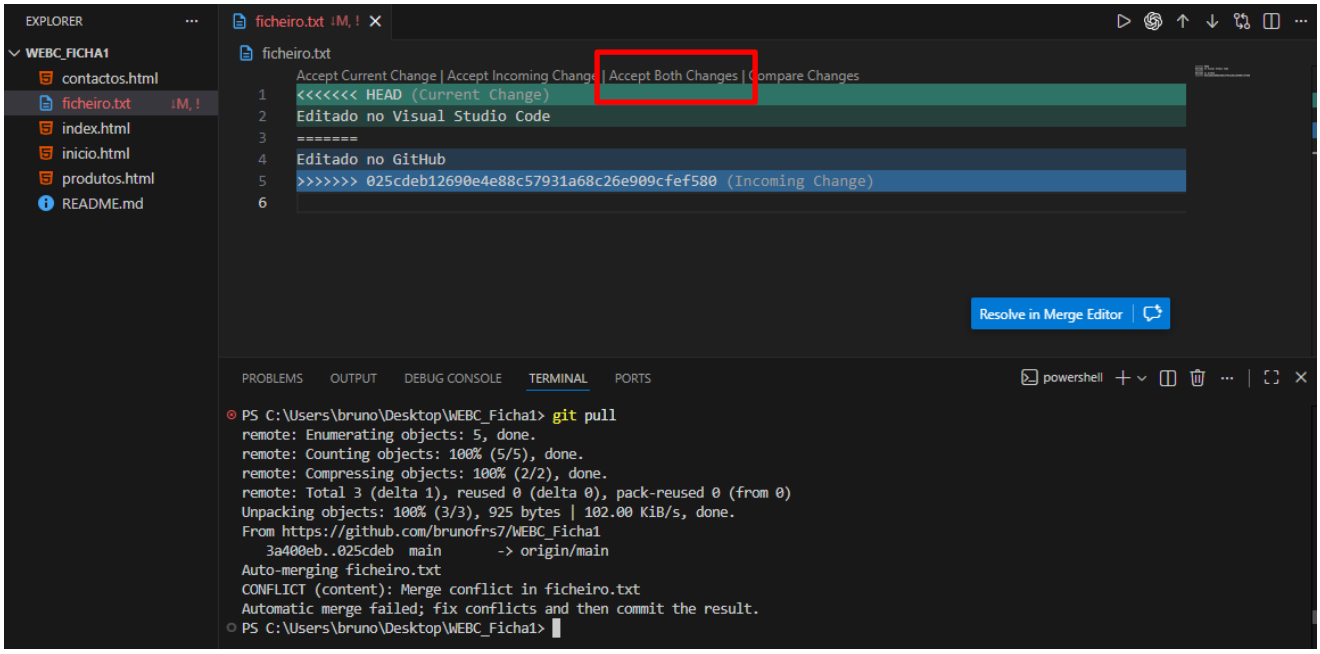


NOTA: este é processo que foge da forma correta de trabalhar, nunca fazemos alterações diretamente no GitHub e principalmente nunca fazemos commit diretamente para a main.

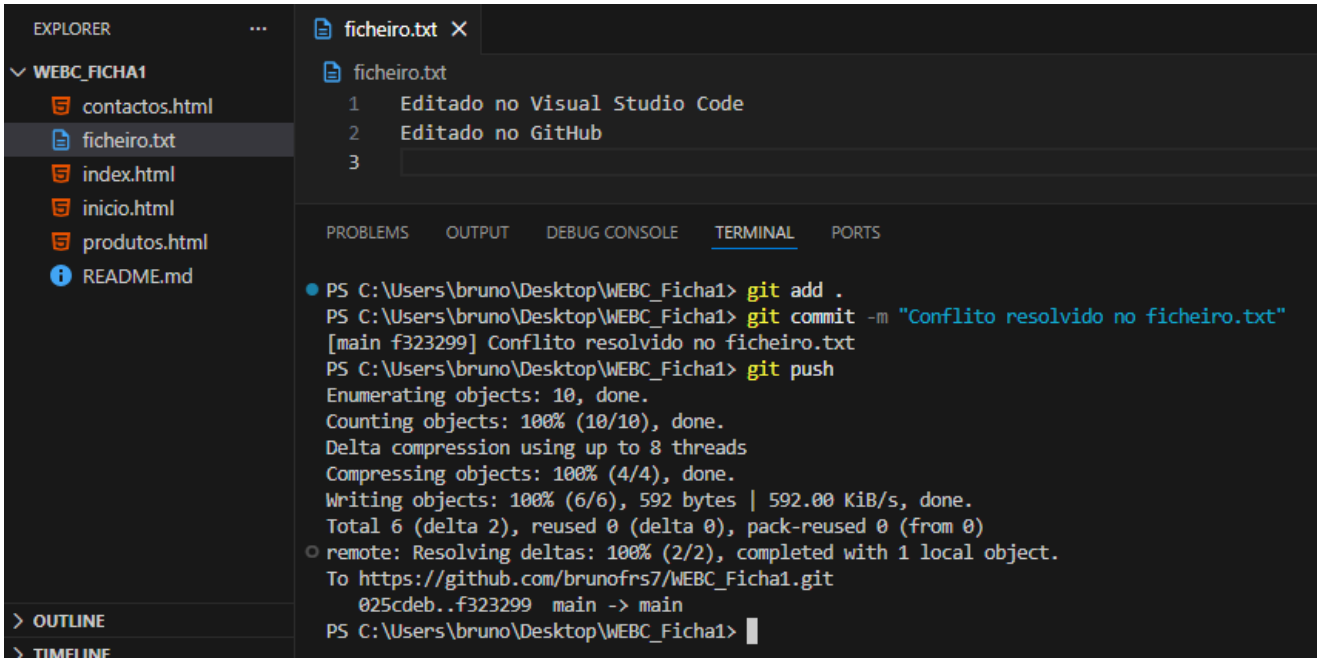
3. No Visual Studio Code edite o mesmo ficheiro colocando "Editado no Visual Studio Code".
4. Abra o terminal e grave as alterações. Utilize como mensagem de commit "ficheiro.txt alterado".
5. Até este ponto não há qualquer erro, mas o conteúdo no GitHub e no repositório local estão diferentes. Vamos tentar enviar o repositório local para o GitHub com o comando "git push".

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git add .
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git commit -m "ficheiro.txt alterado"
[main 3d57e41] ficheiro.txt alterado
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git push
To https://github.com/brunofrs7/WEBC_Ficha1.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/brunofrs7/WEBC_Ficha1.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

6. Sempre que estamos a trabalhar num repositório partilhado devemos descarregar o repositório online antes de enviar as nossas alterações, assim fazemos primeiro “git pull”.
7. É-nos apresentada uma mensagem de conflito e apresentado o mesmo. Com o Visual Studio Code temos a vantagem de poder automaticamente selecionar qual a alteração que queremos que esteja válida: a local, a do repositório ambas (caso não haja sobreposição) ou comparar. Neste caso manter ambas as opções.



8. Após aceitar as alterações devemos fazer nova adição de ficheiros e commit.
9. Para finalizar voltamos a fazer um pull e push de seguida para garantir que as alterações serão gravadas numa versão única.



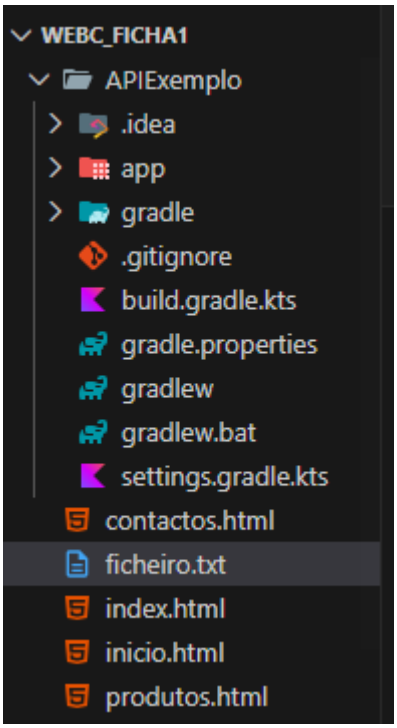
PARTE VII – DESCARREGAR REPOSITÓRIO ONLINE

O GitHub possui um grande número de repositórios públicos, repositórios esses que estão totalmente disponíveis para utilização por parte de qualquer pessoa.

1. No site do GitHub pesquise por um repositório qualquer.
2. No Visual Studio Code, abra o terminal e utilize o comando "git clone LINK" substituindo o LINK pelo link do repositório que pretende descarregar.

```
PS C:\Users\bruno\Desktop\WEBC_Ficha1> git clone https://github.com/brunofrs7/APIExemplo
Cloning into 'APIExemplo'...
remote: Enumerating objects: 102, done.
remote: Counting objects: 100% (102/102), done.
remote: Compressing objects: 100% (73/73), done.
remote: Total 102 (delta 9), reused 101 (delta 8), pack-reused 0 (from 0)
Receiving objects: 100% (102/102), 171.24 KiB | 1.58 MiB/s, done.
Resolving deltas: 100% (9/9), done.
PS C:\Users\bruno\Desktop\WEBC_Ficha1>
```

3. Verifique que na pasta do projeto foi criada uma pasta com o conteúdo do repositório descarregado.



Estas são algumas funcionalidades disponibilizadas pelo Git e GitHub, uma ferramenta utilizada em larga escala pelas empresas e que permite aumentar a produtividade.

EXERCÍCIO - COLABORAÇÃO COM GIT E GITHUB

Objetivos:
No final do exercício, cada formando deverá ser capaz de:

- Clonar um repositório remoto
- Criar e trabalhar numa branch própria
- Editar ficheiros de texto
- Adicionar ficheiros ao projeto
- Fazer commit e push das alterações
- Criar um Pull Request no GitHub
- Resolver conflitos simples (se existirem)

1. Clone o repositório: https://github.com/brunofrs7/exercicio-git-github-colaborativo/
2. Crie uma branch com o seu nome, mude para essa branch e adicione:
    a. Um ficheiro txt com o seu nome como nome do ficheiro e uma pequena biografia como conteúdo.
    b. Altere o README.md adicionando o seu nome na lista de participantes.
    c. Insira uma imagem sua com o seu nome na pasta img.
3. Pesquise sobre GitIgnore, procure saber para que serve, como aplicar e a sua utilidade. Acrescente no seu ficheiro txt uma linha/parágrafo com uma explicação do comando.
4. Após realizar estas alterações faça commit nas mesmas e push para o GitHub. Caso existam conflitos aceite ambas as alterações.
5. No GitHub crie um pull request para que as suas alterações sejam aceites.

Bom trabalho! ☺